Universidade Estadual da Paraíba – UEPB
Disciplina: Linguagem de Programação II

Encapsulamento e Métodos Estáticos em Java

Parte 1: Encapsulamento

O problema: atributos públicos são perigosos

Nos exemplos anteriores, declaramos atributos diretamente na classe, sem nenhuma proteção. Isso permite que **qualquer código externo** leia e altere esses dados sem qualquer controle.

```
1  public class ContaBancaria {
2      String titular;
3      double saldo;  // sem protecao!
4  }
5
6  public class Main {
7      public static void main(String[] args) {
8          ContaBancaria conta = new ContaBancaria();
9          conta.saldo = -9999.0; // PROBLEMA: ninguem deveria fazer
       isso!
10     }
11 }
```

Listing 1: Atributos públicos: qualquer um pode alterar

Por que isso é perigoso?

Sem controle de acesso, qualquer parte do código pode:

- Colocar um **saldo negativo** diretamente em uma conta.
- Alterar o **nome** de um aluno sem passar por nenhuma validação.
- Deixar um objeto em um estado **inconsistente**, quebrando a lógica do programa.

Um bom programa deve proteger os dados internos de um objeto.

O que é encapsulamento?

Definição: Encapsulamento

Encapsulamento é a prática de **esconder os atributos** de um objeto, tornando-os privados (`private`), e fornecer **métodos públicos** (`getters` e `setters`) como a única forma controlada de acessá-los ou modificá-los. O objeto protege seus próprios dados.

Analogia: o caixa eletrônico

Você não tem acesso direto ao cofre do banco. Para depositar ou sacar dinheiro, você usa o **painel do caixa eletrônico** — que verifica se a operação é válida antes de executá-la. Os atributos são o cofre (`private`). Os métodos são o painel (interface pública).

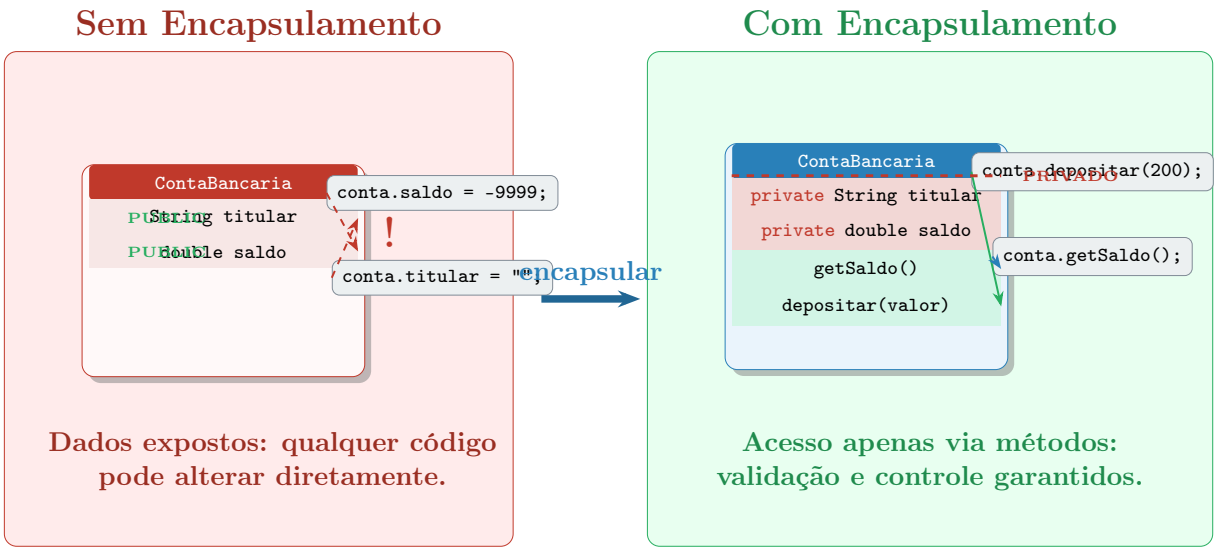


Figure 1: Sem encapsulamento, atributos ficam expostos a alterações indevidas. Com encapsulamento, o acesso passa obrigatoriamente pelos métodos da classe.

Os modificadores `private` e `public`

Definição: Modificadores de acesso

- `private`: o membro (atributo ou método) só pode ser acessado **dentro da própria classe**.
- `public`: o membro pode ser acessado por **qualquer outra classe** do programa.

Regra geral: atributos `private`, métodos de acesso `public`.

Os quatro modificadores de acesso em Java

Em Java existem quatro níveis de visibilidade. Por ora, os mais importantes para o encapsulamento são `private` e `public`, mas a tabela completa ajuda a entender o espectro de controle disponível.

Modificador	Mesma classe	Mesmo pacote	Subclasse (herança)	Qualquer classe
`public`	Sim	Sim	Sim	Sim
`protected`	Sim	Sim	Sim	Não
(sem modificador)	Sim	Sim	Não	Não
`private`	Sim	Não	Não	Não

O que usar agora?

No início, use apenas `private` para atributos e `public` para métodos e construtores. Os modificadores `protected` e *(sem modificador)* serão explorados quando você estudar **herança** e **pacotes**.

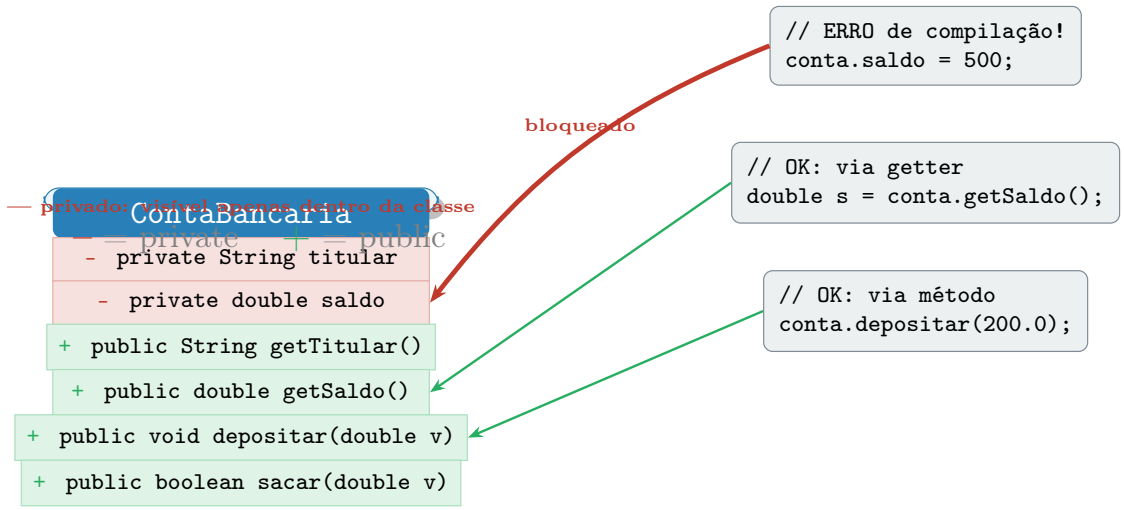


Figure 2: Notação UML: `-` indica `private` (bloqueado ao exterior) e `+` indica `public` (acessível ao exterior).

Getters e Setters

Definição: Getter e Setter

- **Getter** (`getXxx`): método `public` que **retorna** o valor de um atributo `private`.
- **Setter** (`setXxx`): método `public` que **altera** o valor de um atributo `private`, podendo validar o dado antes.

Quando criar getter e setter?

Nem todo atributo precisa de um setter. Pergunte: **"faz sentido alterar este dado de fora da classe?"** Por exemplo, o saldo de uma conta não deve ser alterado diretamente — ele só muda via `depositar()` ou `sacar()`. Crie setters apenas quando a modificação externa for realmente necessária.

Construtor e setter: evite duplicar a validação (e evite chamar o setter dentro do construtor)

O construtor e o setter de um mesmo atributo costumam precisar **da mesma regra de validação**. Para não escrever essa regra duas vezes, um instinto comum é chamar o próprio setter de dentro do construtor:

Evite: construtor chamando o setter público

```
public class ContaBancaria {
    private String titular;

    public ContaBancaria(String titular) {
        setTitular(titular); // parece pratico, mas e uma ma
   pratica
    }

    public void setTitular(String novoTitular) {
        if (novoTitular != null && !novoTitular.isEmpty()) {
            this.titular = novoTitular;
        }
    }
}
```

Isso funciona neste exemplo simples, mas é considerado má prática em Java por dois motivos:

- O construtor deveria ser responsável apenas por **inicializar** o objeto; chamar um método `public` de dentro dele mistura essa responsabilidade com a interface externa da classe.
- Métodos `public` podem ser **sobrescritos** por uma subclasse (você verá isso ao estudar herança). Se isso acontecer, o construtor pode acabar executando uma versão do setter que depende de atributos que **ainda não foram inicializados** — um erro raro e difícil de depurar.

Solução: método privado auxiliar de validação

A prática recomendada é criar um método `private` que recebe o valor **bruto** e devolve o valor **já validado** (corrigido para um padrão seguro, se necessário). Como esse método já devolve o valor pronto para ser gravado, o construtor e o setter podem chamá-lo **da mesma forma, com a mesma linha de código** — é o reaproveitamento de código funcionando na prática. Um método `private` não faz parte da interface pública da classe e não pode ser sobrescrito, então é seguro chamá-lo tanto

no construtor quanto no setter.

```java
public class ContaBancaria {
    private String titular;

    public ContaBancaria(String titular) {
        // o construtor reaproveita o metodo de validacao
        this.titular = validaTitular(titular);
    }

    // metodo PRIVADO: recebe o valor bruto e devolve o valor ja validado
    private String validaTitular(String titular) {
        if (titular != null && !titular.isEmpty()) {
            return titular;
        }
        return "Nao informado"; // valor padrao usado quando o dado e invalido
    }

    public void setTitular(String novoTitular) {
        // o setter reaproveita EXATAMENTE a mesma linha do construtor
        this.titular = validaTitular(novoTitular);
    }
}
```

Listing 2: Padrão correto: método privado de validação reaproveitado

Regra prática

Sempre que o construtor e um setter precisarem da **mesma regra** de validação, crie um método `private` no formato `validaXxx(valor)` que **retorna o valor já corrigido**. Assim, tanto o construtor quanto o setter se resumem a uma única linha: `this.atributo = validaXxx(valor)`. Métodos auxiliares como esse devem ser sempre `private`: eles existem apenas para apoiar a própria classe, e não fazem sentido fora dela.

```java
public class ContaBancaria {

    // Atributos PRIVADOS: ninguem de fora acessa diretamente
    private String titular;
    private int numero;
    private double saldo;

    // Construtor: inicializa o objeto reaproveitando o metodo de validacao
    public ContaBancaria(String titular, int numero, double saldoInicial) {
        this.titular = validaTitular(titular);
        this.numero  = numero;
```

```
12          this.saldo   = (saldoInicial >= 0) ? saldoInicial : 0;
13      }
14
15      // --- MÉTODO PRIVADO DE VALIDAÇÃO: reaproveitado pelo
    construtor e pelo setter ---
16
17      private String validaTitular(String titular) {
18          if (titular != null && !titular.isEmpty()) {
19              return titular;
20          }
21          return "Nao informado";
22      }
23
24      // --- GETTERS: retornam o valor dos atributos ---
25
26      public String getTitular() {
27          return titular;
28      }
29
30      public int getNumero() {
31          return numero;
32      }
33
34      public double getSaldo() {
35          return saldo;
36      }
37
38      // --- SETTER com validação: chama o MESMO método auxiliar do
    construtor ---
39
40      public void setTitular(String novoTitular) {
41          this.titular = validaTitular(novoTitular);
42      }
43
44      // --- Métodos de negócio: controlam como o saldo muda ---
45
46      public void depositar(double valor) {
47          if (valor > 0) {
48              saldo += valor;
49          }
50      }
51
52      public boolean sacar(double valor) {
53          if (valor > 0 && valor <= saldo) {
54              saldo -= valor;
55              return true;
56          }
57          return false;
58      }
59
60      public void mostrarDados() {
```

```
61          System.out.println("Titular: " + titular);
62          System.out.println("Número:  " + numero);
63          System.out.println("Saldo:   R$ " + saldo);
64      }
65  }
```

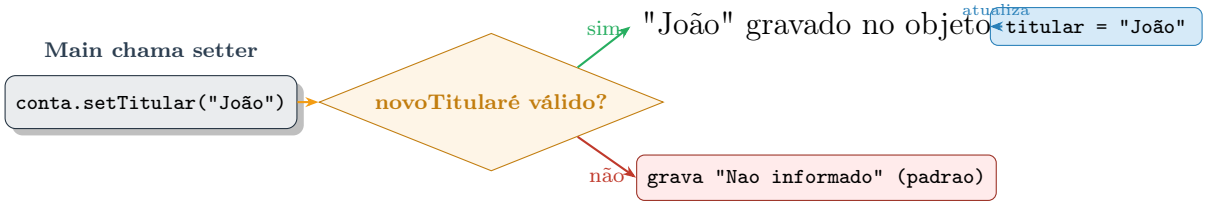
Listing 3: Classe ContaBancaria com encapsulamento completo

```
1  public class Main {
2      public static void main(String[] args) {
3          ContaBancaria conta = new ContaBancaria("Maria", 1001,
       500.0);
4
5          // Lendo dados via getters
6          System.out.println(conta.getTitular()); // "Maria"
7          System.out.println(conta.getSaldo());   // 500.0
8
9          // Modificando via métodos controlados
10         conta.depositar(200.0);
11         conta.sacar(100.0);
12         conta.mostrarDados();
13
14         // Tentativa inválida -- erro de compilação:
15         // conta.saldo = -9999; // não compila!
16     }
17 }
```

Listing 4: Usando a classe encapsulada no main

Verifique seu entendimento

Por que o código `conta.saldo = -9999;` causa erro de compilação com a versão encapsulada da classe, mas não causava antes?



O setter reaproveita o mesmo método de validação do construtor.

Figure 3: Fluxo de um setter com validação: o mesmo método de validação usado no construtor corrige o valor antes de gravá-lo no atributo.

Exemplo completo: classe Aluno encapsulada

```
1  public class Aluno {
2
3      private String nome;
```

```
    private int matricula;
    private double nota1;
    private double nota2;

    public Aluno(String nome, int matricula, double nota1, double nota2) {
        this.nome      = nome;
        this.matricula = matricula;
        // o construtor reaproveita o mesmo metodo de validacao dos setters
        this.nota1 = validaNota(nota1);
        this.nota2 = validaNota(nota2);
    }

    // Método PRIVADO: recebe a nota bruta e devolve a nota ja validada
    private double validaNota(double nota) {
        if (nota >= 0 && nota <= 10) {
            return nota;
        }
        return 0; // valor padrao usado quando a nota e invalida
    }

    // Getters
    public String getNome()       { return nome; }
    public int    getMatricula()  { return matricula; }
    public double getNota1()      { return nota1; }
    public double getNota2()      { return nota2; }

    // Setters: reaproveitam EXATAMENTE o mesmo método do construtor
    public void setNota1(double nota1) {
        this.nota1 = validaNota(nota1);
    }

    public void setNota2(double nota2) {
        this.nota2 = validaNota(nota2);
    }

    // Métodos de negócio
    public double calcularMedia() {
        return (nota1 + nota2) / 2;
    }

    public boolean estaAprovado() {
        return calcularMedia() >= 7;
    }

    public void mostrarDados() {
        System.out.println("Nome: " + nome + " | Matrícula: " + matricula);
```

```
50          System.out.printf("Média: %.1f | Situação: %s%n",
51              calcularMedia(), estaAprovado() ? "Aprovado" : "
    Reprovado");
52      }
53  }
```

Listing 5: Classe Aluno com encapsulamento

Saída esperada

```
Nome: Maria | Matrícula: 101
Média: 8.0 | Situação: Aprovado
```

Tabela resumo: sem vs com encapsulamento

	Sem Encapsulamento	**Com Encapsulamento**
Atributos	`public` (expostos)	`private` (protegidos)
Alteração de dados	Direta: `conta.saldo = x`	Via método: `conta.depositar(x)`
Validação	Nenhuma	Feita dentro do método
Risco de erro	Alto: qualquer valor é aceito	Baixo: o objeto se protege
Manutenção	Difícil	Fácil: muda só a classe

Erros comuns com encapsulamento

1. **Declarar atributo como `public`.** Quebra o encapsulamento e expõe o dado.
2. **Criar setter sem validação.** Um setter que aceita qualquer valor não protege nada.
3. **Retornar referência de objeto mutável no getter.** Para atributos simples (int, double, String) isso não é problema.
4. **Confundir getter com exibição.** Getter apenas retorna o valor — quem exibe é quem chama.

Verifique seu entendimento

Se um setter de nota não tiver nenhuma validação, qual seria o perigo? Tente criar um exemplo de chamada que geraria um estado inválido.

Cuidado: getter e setter para tudo fere o encapsulamento

Um erro muito comum em iniciantes é criar automaticamente um getter **e** um setter para **cada** atributo privado. Isso parece organizado, mas na prática **desfaz o encapsulamento**: se qualquer atributo pode ser lido e alterado livremente de fora, o `private` não serve de nada.

O falso encapsulamento

```java
public class ContaBancaria {
    private double saldo;

    public double getSaldo()           { return saldo; }
    public void   setSaldo(double s)   { this.saldo = s; } // PROBLEMA!
}

// Na prática, isso é igual a deixar o atributo public:
conta.setSaldo(-9999); // nenhuma validação, nenhuma proteção
```

Criar um setter sem lógica alguma é o mesmo que deixar o atributo público. O encapsulamento existe para **proteger**, não apenas para esconder.

Analogia: o cofre com a senha colada na porta

De que adianta trancar o cofre se a senha está escrita num post-it colado na própria porta? Um setter sem validação é exatamente isso: o atributo continua acessível a qualquer valor, só com um passo a mais.

A pergunta certa ao decidir se um atributo precisa de getter ou setter é:

Pergunta	**Resposta**	**O que criar**
O exterior precisa **ler** este dado?	Sim	Crie o **getter**
O exterior precisa **alterar** este dado diretamente?	Raramente	Crie setter **com validação**
O dado só muda por **regras de negócio**?	Sim	Crie métodos específicos (`depositar`, `sacar`)
Nem leitura nem escrita externa fazem sentido?	Sim	**Não crie** getter nem setter

```java
public class ContaBancaria {
    private String titular;
    private double saldo;
    private int    numeroDeTentativasFalhas; // interno; ninguem precisa ver

    // Getter faz sentido: quem usa a conta precisa saber o saldo
    public double getSaldo()   { return saldo; }

    // Getter faz sentido: titular pode ser exibido
    public String getTitular() { return titular; }

    // SEM setter de saldo: ele só muda por regras de negócio
```

```
    public void depositar(double valor) {
        if (valor > 0) saldo += valor;
    }
    public boolean sacar(double valor) {
        if (valor > 0 && valor <= saldo) { saldo -= valor; return
   true; }
        return false;
    }

    // SEM getter de tentativas: é detalhe interno da classe
}
```

Listing 6: Exemplo correto: só o necessário é exposto

Verifique seu entendimento

Olhando a classe `Aluno` que criamos antes, quais atributos realmente precisam de setter? O atributo `matricula` deveria ter um setter? Por quê?

Parte 2: Métodos Estáticos

O problema: métodos que não pertencem a nenhum objeto

Até agora, todos os métodos que vimos pertencem a **objetos**: para chamar `calcularMedia()`, você precisa ter um objeto `Aluno`. Mas às vezes precisamos de funções que **não dependem de nenhum objeto específico** — elas apenas recebem valores, processam e retornam um resultado.

```
// Para calcular a média de qualquer par de notas,
// seria estranho precisar criar um objeto só para isso:
Aluno a = new Aluno(...); // criar um objeto só para calcular?
double m = a.calcularMedia(); // não faz sentido aqui!

// O ideal seria chamar diretamente:
double m = Matematica.media(8.0, 9.0); // mais claro e direto
```

Listing 7: Situação: precisamos de funções de cálculo genéricas

O que é um método estático?

Definição: Método Estático

Um **método estático** (declarado com a palavra `static`) pertence **à classe**, não a objetos individuais. Ele pode ser chamado pelo **nome da classe**, sem precisar criar um objeto primeiro.

Analogia: a calculadora

Pense em uma calculadora de mesa. Você não precisa "criar" uma calculadora para usar cada conta — você simplesmente a usa. Métodos estáticos são como as funções da calculadora: independem de um objeto particular, apenas recebem números e

devolvem resultados.

```java
public class Matematica {

    // Método estático: pertence à CLASSE, não a objetos
    public static double media(double a, double b) {
        return (a + b) / 2;
    }

    public static double maior(double a, double b) {
        return (a > b) ? a : b;
    }
}

// Chamada: usa o NOME DA CLASSE, sem criar objeto
double resultado = Matematica.media(8.0, 9.0);    // 8.5
double m = Matematica.maior(7.0, 9.5);             // 9.5
```

Listing 8: Sintaxe básica de um método estático

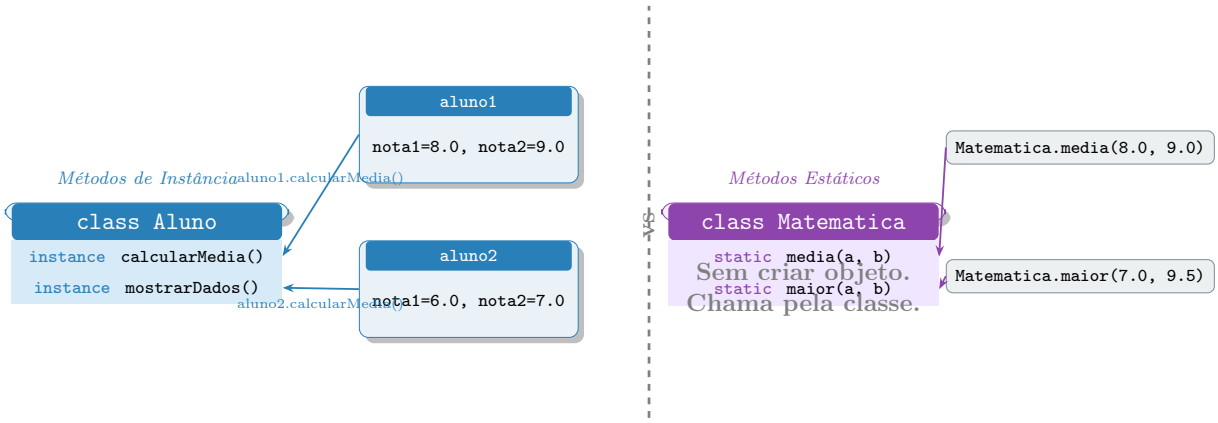


Figure 4: Métodos de instância exigem um objeto; métodos estáticos são chamados diretamente pelo nome da classe.

A regra de ouro do `static`

Regra: o que um método estático pode usar

Um método estático **não tem acesso** aos atributos de instância do objeto (pois não existe objeto quando ele é chamado). Ele só pode:

- Usar os **parâmetros** que recebeu.
- Chamar outros **métodos estáticos**.
- Acessar **atributos estáticos** (variáveis da classe, não do objeto).

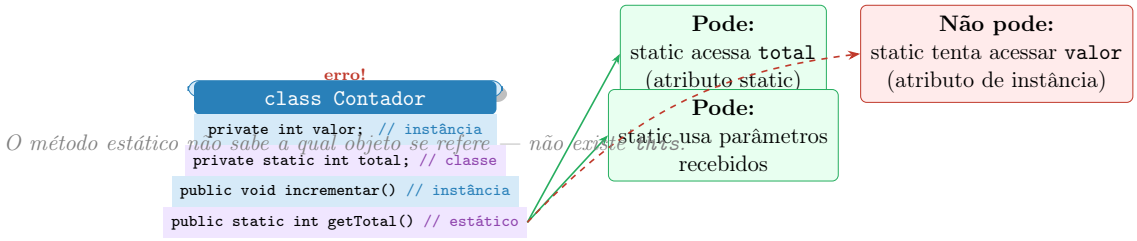


Figure 5: Método estático pode acessar atributos e métodos estáticos, mas não atributos de instância (pois não há objeto).

Exemplo: classe de utilidades matemáticas

```java
public class Matematica {

    // Métodos utilitários: recebem parâmetros, não dependem de objeto

    public static double media(double nota1, double nota2) {
        return (nota1 + nota2) / 2.0;
    }

    public static double maior(double a, double b) {
        return (a > b) ? a : b;
    }

    public static boolean estaAprovado(double media) {
        return media >= 7.0;
    }

    public static String situacao(double media) {
        if (media >= 7.0)       return "Aprovado";
        else if (media >= 5.0)  return "Recuperação";
        else                    return "Reprovado";
    }
}
```

Listing 9: Classe Matematica com métodos estáticos

```java
public class Main {
    public static void main(String[] args) {

        double nota1 = 8.0, nota2 = 6.5;

        // Chamada sem criar objeto: NomeClasse.metodo(...)
        double media = Matematica.media(nota1, nota2);
        System.out.println("Média: " + media);                    // 7.25
        System.out.println("Situação: " + Matematica.situacao(media)); // Aprovado
        System.out.println("Maior nota: " + Matematica.maior(nota1, nota2)); // 8.0
```

```
    }
}
```

Listing 10: Usando os métodos estáticos diretamente

Saída esperada

```
Média: 7.25
Situação: Aprovado
Maior nota: 8.0
```

Atributo estático: contando objetos criados

Definição: Atributo Estático

Um **atributo estático** (declarado com `static`) pertence à **classe**, não a cada objeto individualmente. Todos os objetos **compartilham** o mesmo valor. É útil para guardar informações sobre a classe como um todo, como a quantidade de objetos criados.

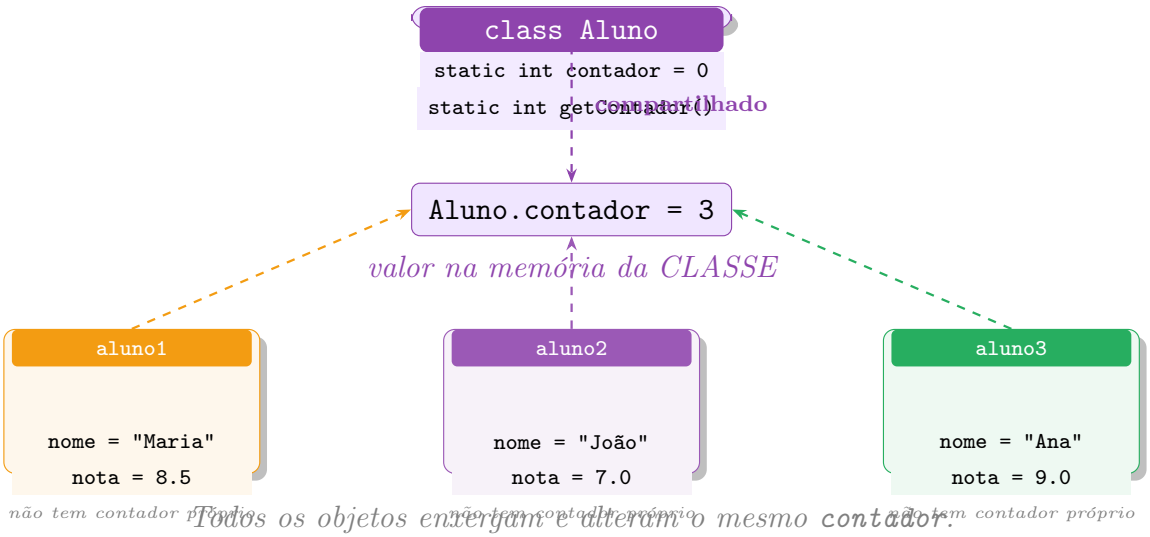


Figure 6: Atributo estático: um único valor compartilhado por todos os objetos da classe. Cada objeto tem seus próprios atributos de instância, mas o atributo `static` é da classe.

```
public class Aluno {

    private String nome;
    private double nota;

    // Atributo ESTÁTICO: pertence à CLASSE, não aos objetos
    private static int contador = 0;

    public Aluno(String nome, double nota) {
        this.nome = nome;
        this.nota = nota;
```

```
        contador++; // incrementa a cada novo objeto criado
    }

    public String getNome()  { return nome; }
    public double getNota()  { return nota; }

    // Método ESTÁTICO: retorna o contador da classe
    public static int getContador() {
        return contador;
    }
}
```

Listing 11: Classe Aluno com contador estático

```
public class Main {
    public static void main(String[] args) {

        System.out.println(Aluno.getContador()); // 0

        Aluno a1 = new Aluno("Maria", 8.5);
        Aluno a2 = new Aluno("João",  7.0);
        Aluno a3 = new Aluno("Ana",   9.0);

        System.out.println(Aluno.getContador()); // 3

        // Cada objeto tem seus PRÓPRIOS dados:
        System.out.println(a1.getNome()); // Maria
        System.out.println(a2.getNome()); // João
    }
}
```

Listing 12: Usando o atributo estático no main

Saída esperada

```
0
3
Maria
João
```

Verifique seu entendimento

Se você criar mais 2 objetos `Aluno` depois do código acima, qual será o valor retornado por `Aluno.getContador()`?

Combinando encapsulamento e static: exemplo completo

Vamos criar um sistema de alunos que usa encapsulamento para proteger os dados e métodos estáticos para operações independentes de objetos.

```
public class Aluno {

    // Atributos de instância (privados)
```

```
4     private String nome;
5     private int    matricula;
6     private double nota1;
7     private double nota2;
8
9     // Atributo estático (da classe, não do objeto)
10    private static int totalAlunos = 0;
11
12    // Construtor
13    public Aluno(String nome, int matricula, double nota1, double
    nota2) {
14        this.nome      = nome;
15        this.matricula = matricula;
16        // o construtor reaproveita o mesmo metodo de validacao dos
     setters
17        this.nota1 = validaNota(nota1);
18        this.nota2 = validaNota(nota2);
19        totalAlunos++;
20    }
21
22    // Método PRIVADO: recebe a nota bruta e devolve a nota ja
    validada
23    private double validaNota(double nota) {
24        if (nota >= 0 && nota <= 10) {
25            return nota;
26        }
27        return 0; // valor padrao usado quando a nota e invalida
28    }
29
30    // Getters
31    public String getNome()       { return nome; }
32    public int    getMatricula()  { return matricula; }
33    public double getNota1()      { return nota1; }
34    public double getNota2()      { return nota2; }
35
36    // Setters: reaproveitam EXATAMENTE o mesmo método do
    construtor
37    public void setNota1(double nota1) {
38        this.nota1 = validaNota(nota1);
39    }
40    public void setNota2(double nota2) {
41        this.nota2 = validaNota(nota2);
42    }
43
44    // Métodos de instância
45    public double calcularMedia() {
46        return (nota1 + nota2) / 2.0;
47    }
48
49    public boolean estaAprovado() {
50        return calcularMedia() >= 7.0;
```

```
    }

    public void mostrarDados() {
        System.out.printf("%-10s | mat: %d | media: %.1f | %s%n",
            nome, matricula, calcularMedia(),
            estaAprovado() ? "Aprovado" : "Reprovado");
    }

    // Métodos ESTÁTICOS: não pertencem a objetos
    public static int getTotalAlunos() {
        return totalAlunos;
    }

    public static double mediaDaTurma(Aluno[] turma) {
        double soma = 0;
        for (Aluno a : turma) {
            soma += a.calcularMedia();
        }
        return soma / turma.length;
    }

    public static Aluno melhorAluno(Aluno[] turma) {
        Aluno melhor = turma[0];
        for (Aluno a : turma) {
            if (a.calcularMedia() > melhor.calcularMedia()) {
                melhor = a;
            }
        }
        return melhor;
    }
}
```

Listing 13: Classe Aluno com encapsulamento + static

```
public class Main {
    public static void main(String[] args) {

        System.out.println("Alunos criados: " + Aluno.
   getTotalAlunos()); // 0

        Aluno[] turma = {
            new Aluno("Maria", 101, 8.0, 9.0),
            new Aluno("João",  102, 6.0, 7.0),
            new Aluno("Ana",   103, 9.5, 9.0),
            new Aluno("Pedro", 104, 5.0, 4.0)
        };

        System.out.println("Alunos criados: " + Aluno.
   getTotalAlunos()); // 4

        System.out.println("\n--- Dados da turma ---");
        for (Aluno a : turma) {
```

```java
            a.mostrarDados();
        }

        System.out.printf("%nMédia da turma: %.1f%n",
            Aluno.mediaDaTurma(turma));

        Aluno melhor = Aluno.melhorAluno(turma);
        System.out.println("Melhor aluno: " + melhor.getNome());
    }
}
```

Listing 14: Main usando tudo junto

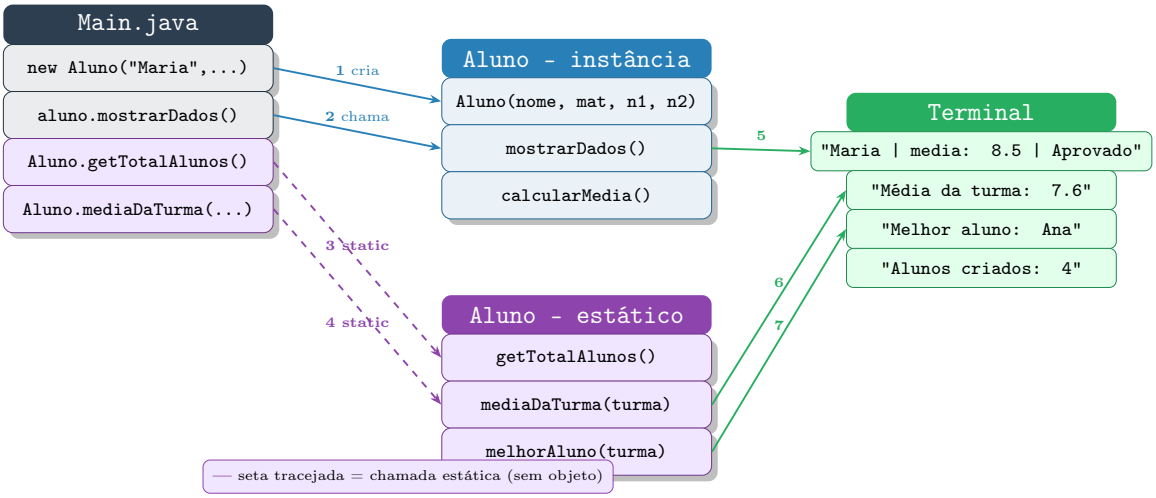


Figure 7: Fluxo completo: setas sólidas representam chamadas em objetos (instância); setas tracejadas representam chamadas estáticas (diretamente na classe).

Saída esperada

```
Alunos criados: 0
Alunos criados: 4

--- Dados da turma ---
Maria      | mat: 101 | media: 8.5 | Aprovado
João       | mat: 102 | media: 6.5 | Reprovado
Ana        | mat: 103 | media: 9.3 | Aprovado
Pedro      | mat: 104 | media: 4.5 | Reprovado

Média da turma: 7.2
Melhor aluno: Ana
```

Quando usar `static` e quando não usar

	Método de Instância	**Método Estático**
Pertence a	Um objeto específico	A classe
Como chama	`objeto.metodo()`	`Classe.metodo()`
Acessa `this`?	Sim	Não
Usa atributos de instância?	Sim	Não
Quando usar	Comportamento do objeto	Utilidades, fábricas, contadores
Exemplos	`calcularMedia()`, `sacar()`	`Math.sqrt()`, `Integer.parseInt()`

Dica prática: como decidir?

Pergunte: **"esse método precisa dos dados de um objeto específico?"**

- **Sim** → método de instância (sem `static`).
- **Não** → considere torná-lo estático.

`calcularMedia()` usa `nota1` e `nota2` do objeto → instância.
`media(a, b)` recebe tudo por parâmetro → estático.

Erros comuns com métodos estáticos

1. **Tentar usar `this` dentro de um método estático.** Causa erro de compilação: "`Cannot use this in a static context`".
2. **Tentar acessar atributo de instância em método estático.** Também não compila: o método não sabe a qual objeto se refere.
3. **Criar objeto só para chamar método estático.** Errado: `new Matematica().media(a,b)`. Correto: `Matematica.media(a,b)`.
4. **Tornar tudo estático.** Se tudo for `static`, você perde os benefícios da POO. Use com critério.

Resumo geral dos conceitos

Conceito	Explicação simples	Pergunta-chave
Encapsulamento	Protege os atributos com `private` e controla o acesso por métodos.	Os dados estão protegidos?
`private`	Visível apenas dentro da própria classe.	Quem pode ver?
`public`	Visível por qualquer classe.	Quem pode usar?
Getter	Retorna o valor de um atributo privado.	Como leio o dado?
Setter	Altera um atributo privado com validação.	Como altero com segurança?
`static` **(método)**	Pertence à classe; chamado sem objeto.	Precisa de objeto?
`static` **(atributo)**	Valor compartilhado por todos os objetos.	É da classe ou do objeto?

Atividade prática

Escolha **um** dos temas abaixo e implemente as classes pedidas, aplicando os conceitos desta aula. Sua implementação deve atender a todos os requisitos da lista de verificação abaixo, independentemente do tema escolhido.

Requisitos obrigatórios (valem para qualquer tema)

- Todos os atributos devem ser `private`.
- Pelo menos **um atributo** deve ter validação tanto no construtor quanto no setter, usando um **método privado auxiliar** de validação — o construtor **nunca** deve chamar o setter.
- Pelo menos **um atributo estático** (`static`), como um contador de objetos criados.
- Pelo menos **dois métodos estáticos** que recebam todos os dados de que precisam via parâmetro (sem usar `this`).
- Use **arrays** (não coleções) para representar grupos de objetos, como nos exemplos desta aula.
- Inclua um método `mostrarDados()` (ou similar) que imprime os dados do objeto de forma legível.

Tema 1. Sistema de produtos em uma loja

- **Atributos sugeridos:** `nome`, `preco`, `quantidadeEmEstoque`.

- **Validação obrigatória:** `preco` não pode ser negativo (método privado `validaPreco`, usado no construtor e em `setPreco`).
- **Métodos estáticos sugeridos:**
  - `calcularPrecoComDesconto(preco, percentual)`
  - `produtoMaisCaro(Produto[] produtos)`
- **Atributo estático sugerido:** `totalDeProdutosCriados`.

Tema 2. Sistema de funcionários em uma empresa

- **Atributos sugeridos:** `nome`, `cargo`, `salario`.
- **Validação obrigatória:** `salario` não pode ser negativo nem zero (método privado `validaSalario`, usado no construtor e em `setSalario`).
- **Métodos estáticos sugeridos:**
  - `calcularImposto(salario, aliquota)`
  - `mediaSalarial(Funcionario[] funcionarios)`
- **Atributo estático sugerido:** `totalDeFuncionarios`.

Tema 3. Sistema de notas de uma turma

- **Atributos sugeridos:** `nome`, `matricula`, `nota1`, `nota2`, `nota3`.
- **Validação obrigatória:** cada nota deve estar entre 0 e 10 (método privado `validaNota`, reaproveitado no construtor e em cada setter de nota).
- **Métodos estáticos sugeridos:**
  - `mediaDaTurma(Aluno[] turma)`
  - `melhorAluno(Aluno[] turma)`
  - `piorAluno(Aluno[] turma)`
- **Atributo estático sugerido:** `totalDeAlunos`.

Tema 4. Sistema de contas bancárias

- **Atributos sugeridos:** `titular`, `numero`, `saldo`.
- **Validação obrigatória:** o saldo inicial não pode ser negativo (método privado `validaSaldoInicial`, usado no construtor). Lembre-se: `saldo` não deve ter setter — ele só muda via `depositar()` e `sacar()`.
- **Métodos estáticos sugeridos:**
  - `calcularRendimento(saldo, taxa)`
  - `contaComMaiorSaldo(contas)`
- **Atributo estático sugerido:** `totalDeContasCriadas`.

Tema 5. Sistema de jogadores em um time

- **Atributos sugeridos:** `nome`, `posicao`, `golsMarcados`, `jogosDisputados`.
- **Validação obrigatória:** `golsMarcados` e `jogosDisputados` não podem ser negativos (método privado `validaQuantidade`, reaproveitado no construtor e nos setters correspondentes).

- **Métodos estáticos sugeridos:**
  - `artilheiro(Jogador[] time)`
  - `mediaDeGolsPorJogo(jogador)`
- **Atributo estático sugerido:** `totalDeJogadores`.

Antes de programar, planeje no papel

- Quais atributos serão `private`? Quais realmente precisam de um setter — e quais não fazem sentido alterar de fora da classe?
- Para o(s) atributo(s) com validação repetida no construtor e no setter, qual será o nome do método `private` auxiliar? Siga o padrão `validaXxx(valor)`, que **retorna o valor já corrigido** — assim o construtor e o setter usam a mesma linha: `this.atributo = validaXxx(valor)`.
- Quais métodos **dependem** de um objeto específico (instância) e quais **não dependem** de nenhum objeto (podem ser `static`)?
- Qual atributo `static` é útil para esta classe, além de um contador de objetos?
- Escreva, antes de implementar, qual deve ser a **saída esperada** do seu `Main` de teste — assim você sabe exatamente o que validar ao final.

Verifique seu entendimento

Depois de implementar sua classe, revise o código: existe alguma chamada de `setXxx(...)` dentro do construtor? Se sim, substitua-a pela chamada ao método privado de validação, como foi feito nos exemplos desta aula.